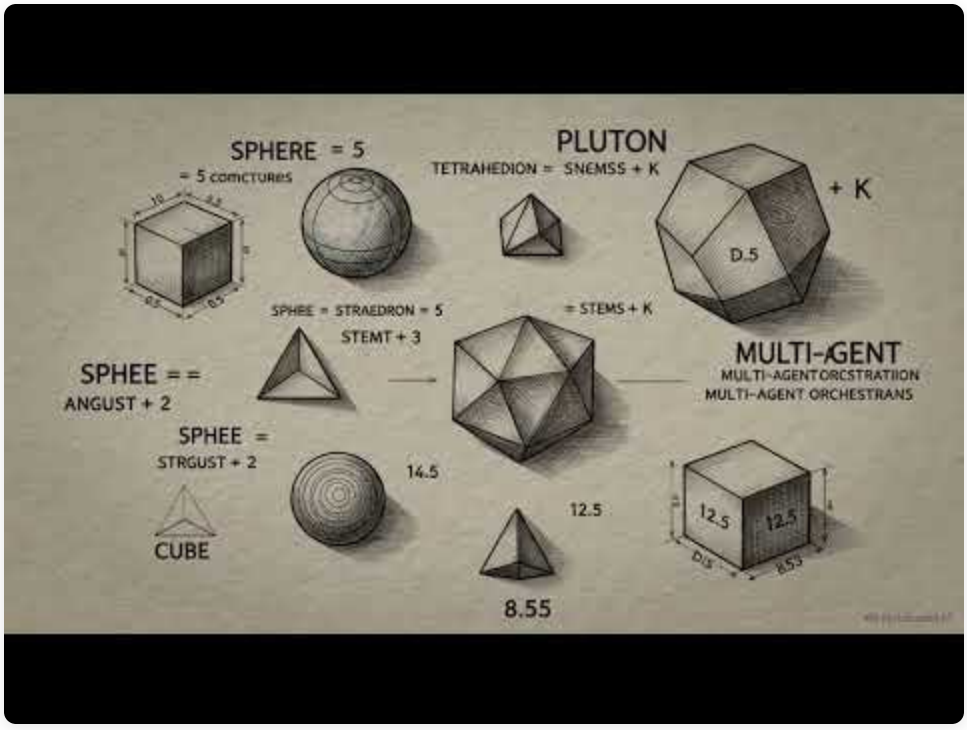
Lesson 1: The Agentic Shift and the Shape of the Cohort

Video Overview



▶ Watch on YouTube: Lesson 1 Video Overview

The Central Thesis

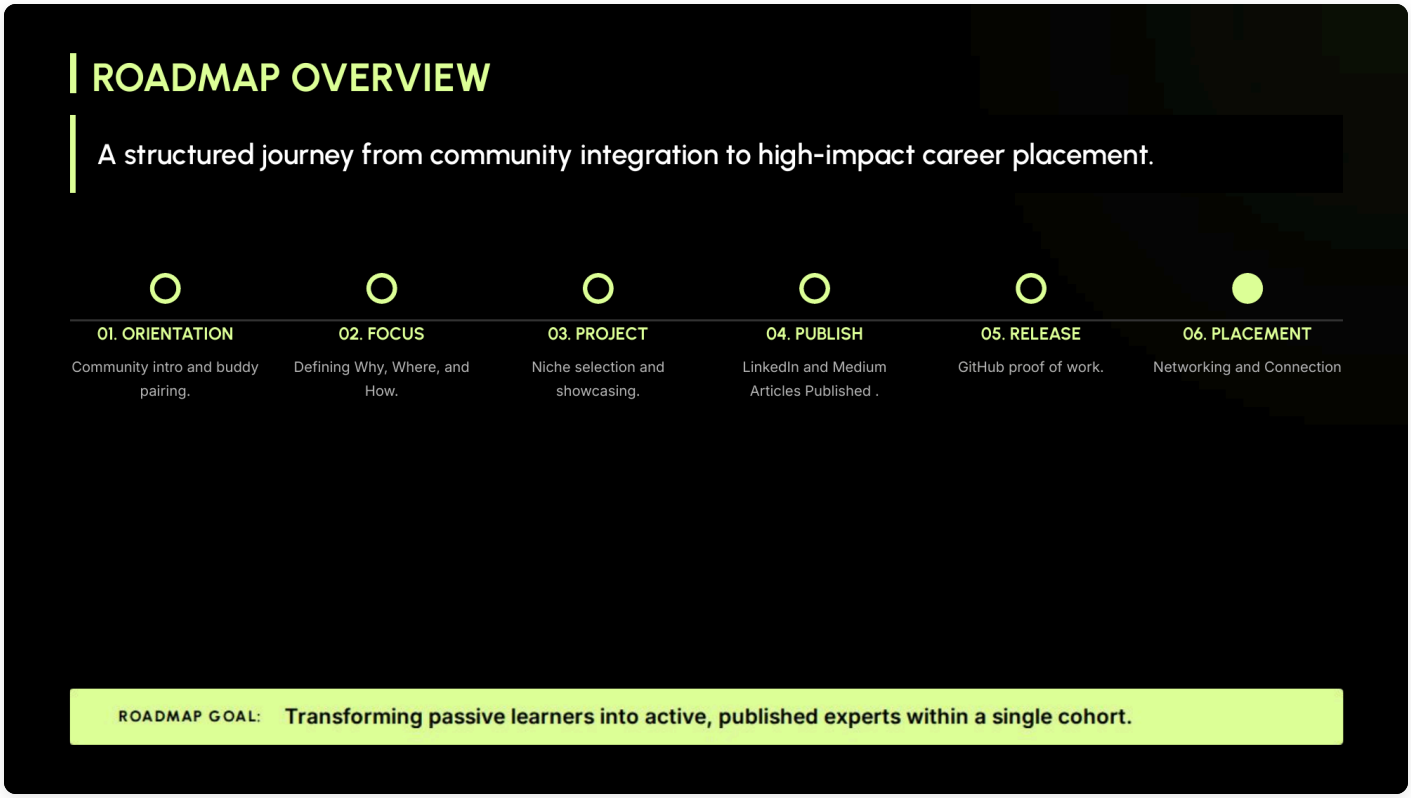
The cohort opens with a single argument: AI engineering has crossed a threshold, and the work that pays now is not the work that paid two years ago. The era of the linear chatbot, a prompt in and a completion out, has collapsed into something more demanding. What sits in front of you is the **Agentic Shift**, the move from request-response systems to self-healing, multi-modal, autonomous agentic systems with measurable ROI.

The program's tagline reflects this: *Bridging the Academic-Enterprise Gap through High-Stakes AI Delivery.* Academic AI courses teach you what a transformer is. Enterprise teams need engineers who can ship an agent that survives a production incident at 2 AM, routes around a failing tool call, observes its own behavior, and stays inside a token budget. That gap is where this cohort plants its flag.

A note on format before the substance: sessions are live and interactive. There are no recordings. If you miss a session, you lean on your buddy pair. This is deliberate. The expectation is that you show up, engage, and treat the cohort as a working community rather than a video library.

The Six-Milestone Roadmap

The cohort structures the journey into six milestones, each one a deliberate transition rather than a checkpoint:



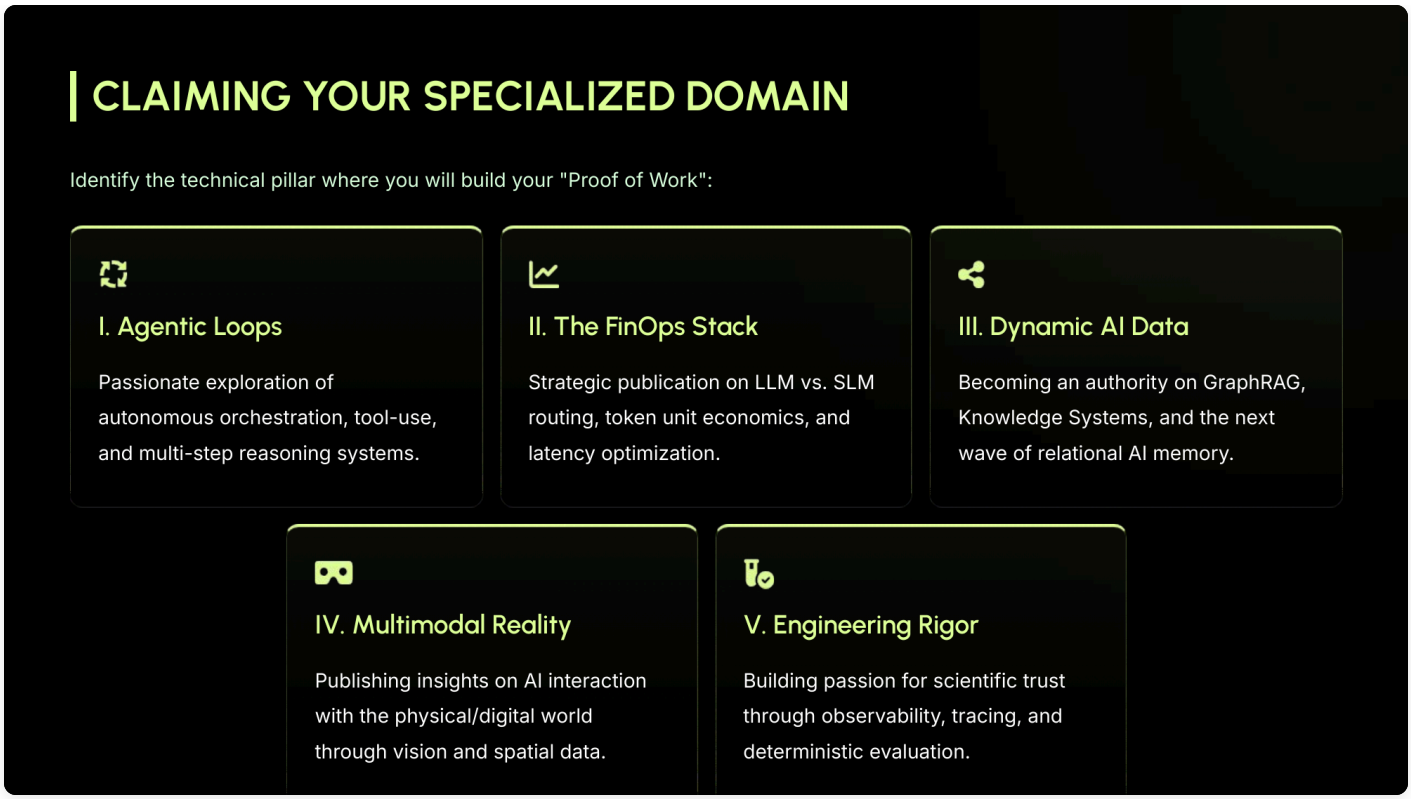
1. **Orientation** — community introduction and buddy pairing. The buddy is your continuity mechanism for missed material and your first peer-review channel.

2. **Focus** — defining your *Why*, *Where*, and *How*. This is where you stop browsing and start narrowing.
3. **Project** — niche selection and showcasing. You pick a domain, a role, and a problem you will be visible on.
4. **Publish** — LinkedIn and Medium articles. Your work goes external, in writing, with your name on it.
5. **Release** — GitHub proof of work. Code, not slides. Architecturally clean, navigable, dogfoodable.
6. **Placement** — networking and connection. The publishing and the release feed into who can find you and why they would want to.

The stated goal of the roadmap is *transforming passive learners into active, published experts within a single cohort.* Read that carefully. The promise is not "you will learn AI." The promise is that by the end you will be a published practitioner with a public artifact trail. The structure exists to force that outcome.

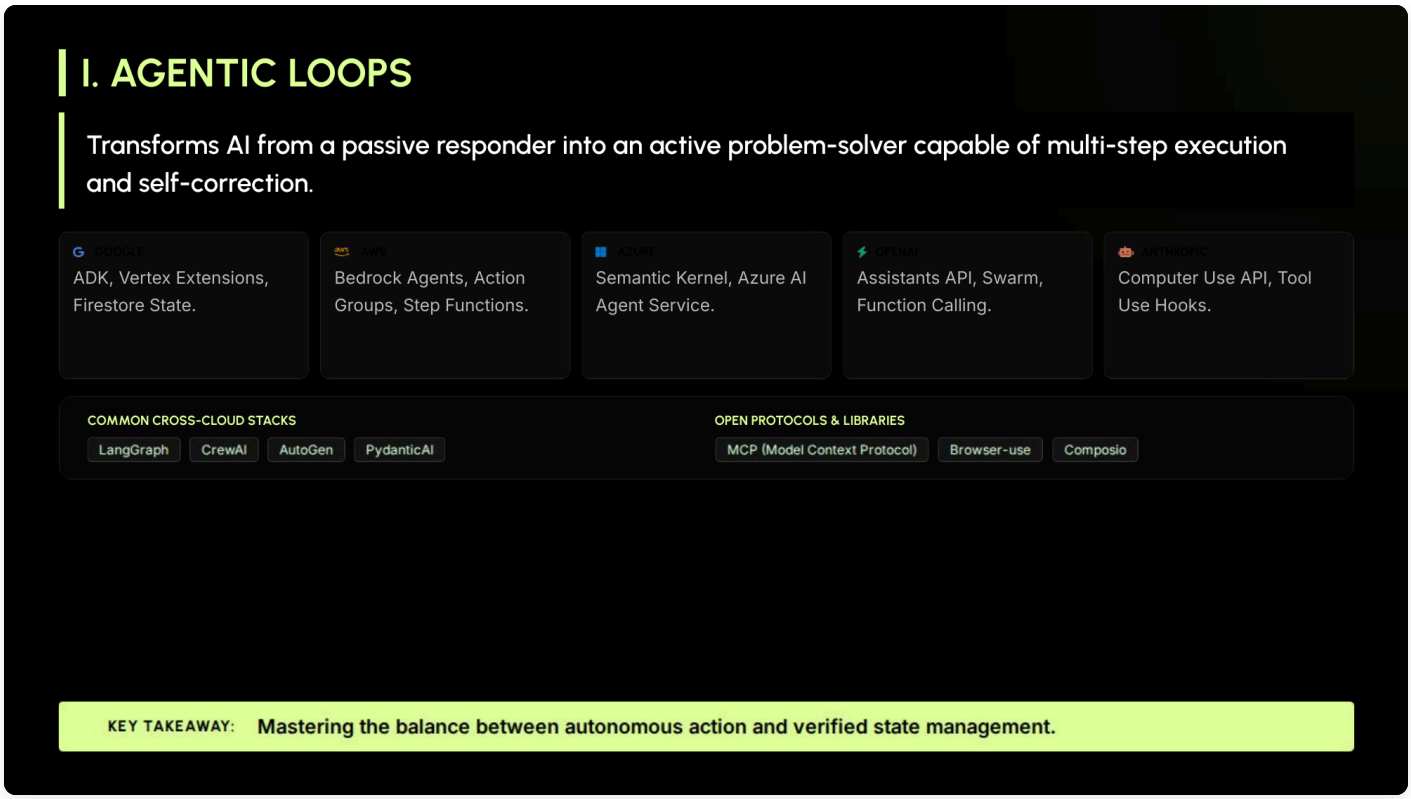
The Five Foundational Concepts

Each student selects one of five foundational concepts as their subject-matter-expert (SME) niche. These are not survey topics; they are pillars deep enough to anchor a body of work around. Pick one and you commit to going past surface level on it.



I. Agentic Loops

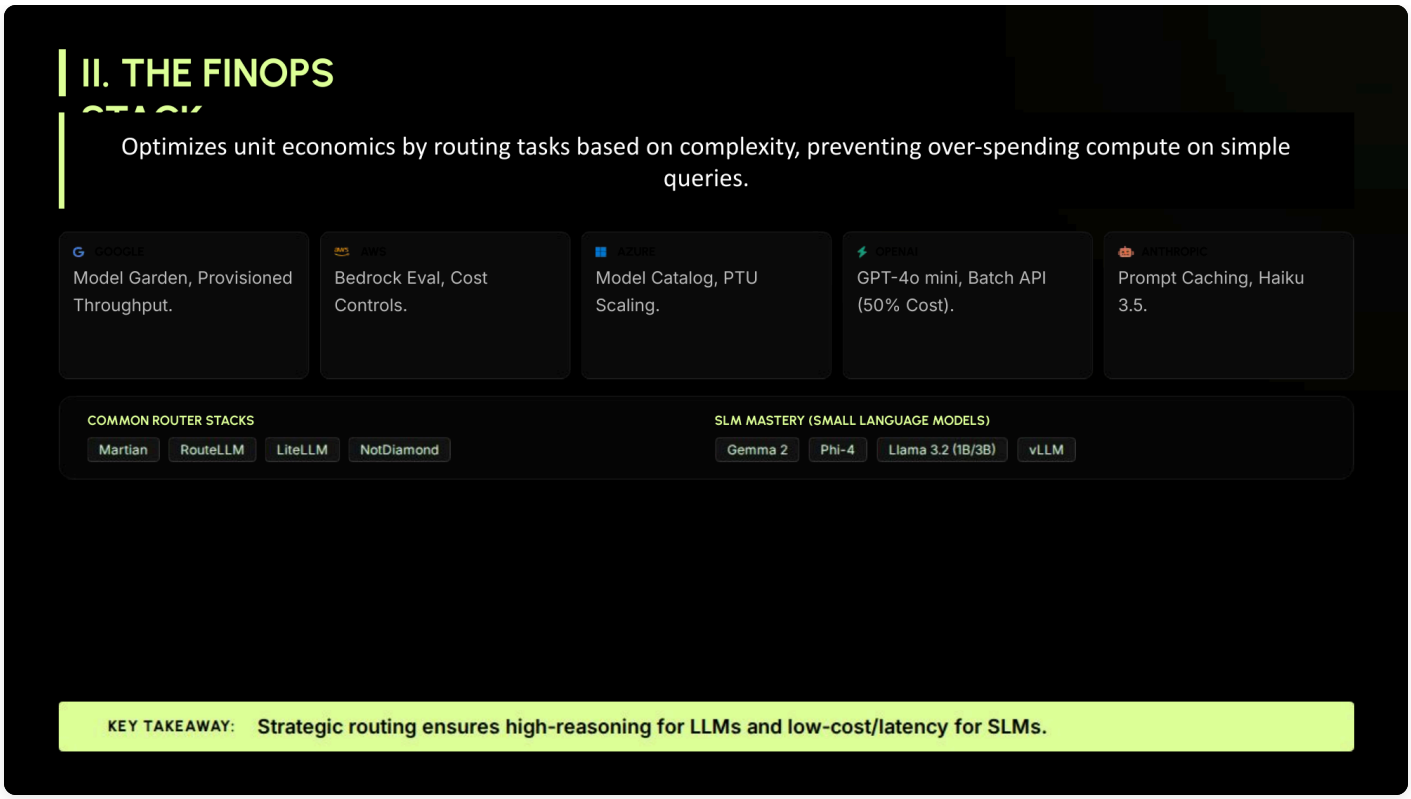
The core pattern: **Plan → Act → Observe → Refine.** This is the orchestration logic that turns an LLM from a text generator into something that can complete a task across many steps. Concretely, this is the territory of ReAct-style reasoning, orchestrator-worker patterns, tool-using agents that decide when to call which function, and multi-step plans that adapt when a step fails. An Agentic Loops specialist understands how to design the planner, what to put in scratch memory, how to structure tool descriptions so the model picks correctly, and how to recover when an Act step returns garbage. The hard problems here are not "can the model use a tool?" but "can the system survive ten tool calls in sequence with one of them failing silently?"



The ecosystem already has mature options across every major cloud: Google's ADK with Vertex Extensions and Firestore for state, AWS Bedrock Agents with Action Groups and Step Functions, Azure's Semantic Kernel and AI Agent Service, OpenAI's Assistants API and Swarm, and Anthropic's Computer Use API and tool-use hooks. Cross-cloud, the open libraries to know are LangGraph, CrewAI, AutoGen, and PydanticAI. The emerging interoperability layer is the Model Context Protocol (MCP), alongside Browser-Use and Composio. **Key takeaway: master the balance between autonomous action and verified state management.**

II. The FinOps Stack

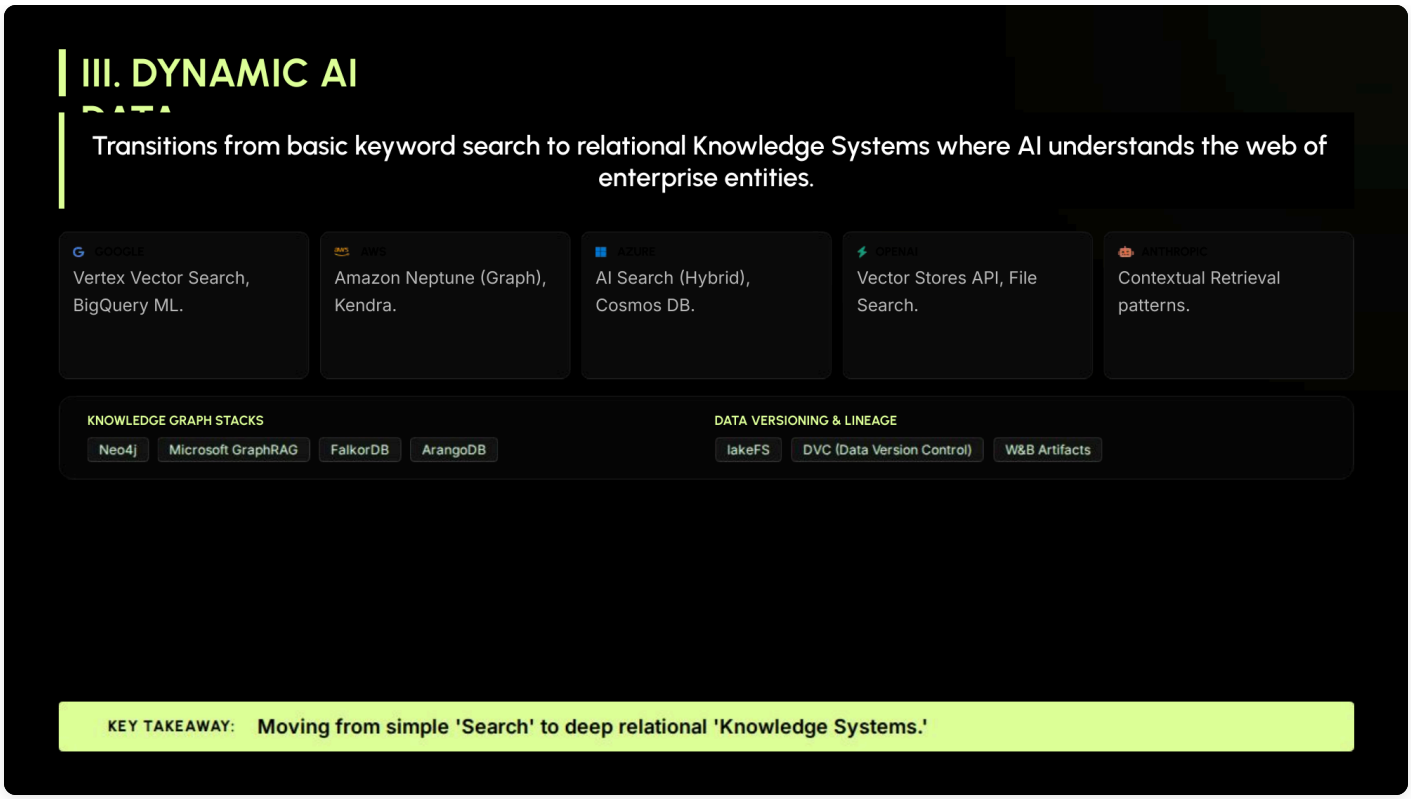
Token economics is the silent killer of AI products. The FinOps stack covers everything that makes an AI system financially viable at scale: choosing between a frontier LLM and a small specialized model (SLM) per call, prompt caching to avoid re-paying for the same context, model routing to send easy queries to cheap models and hard queries to expensive ones, batching to amortize fixed costs, and latency optimization because slow agents lose users regardless of how clever they are. A FinOps specialist can look at a system architecture and tell you the cost-per-task, where the spend is concentrated, and which calls are over-provisioned. This is the discipline that separates a demo from a sustainable product.



The vendor surface for this concept includes Google's Model Garden and Provisioned Throughput, AWS Bedrock's Eval and cost controls, Azure's Model Catalog with PTU scaling, OpenAI's GPT-4o mini and 50%-cost Batch API, and Anthropic's prompt caching plus Haiku 3.5. Common router stacks include Martian, RouteLLM, LiteLLM, and NotDiamond. For SLM mastery, the working set is Gemma 2, Phi-4, Llama 3.2 (1B/3B), and the vLLM serving runtime. **Key takeaway: strategic routing ensures high-reasoning for LLMs and low-cost / low-latency for SLMs.**

III. Dynamic AI Data

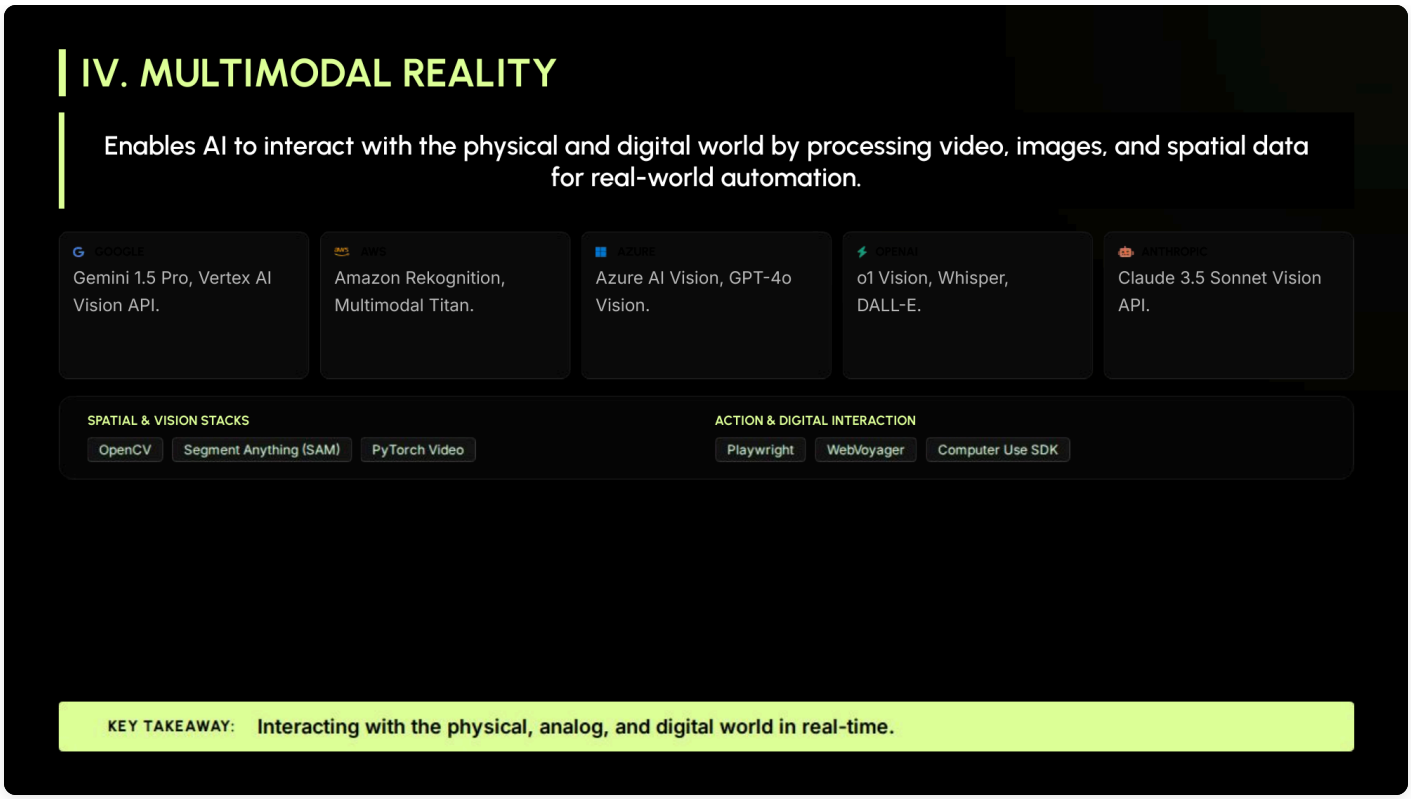
This is the data layer for agents that need memory and structured knowledge. **GraphRAG** is the headline term: rather than chunking documents and retrieving by vector similarity alone, you build a knowledge graph over the source material, so retrieval can traverse relationships, not just match embeddings. Entities, relations, and the paths between them become first-class. This pillar also covers relational AI memory more broadly: how an agent remembers what happened across sessions, how it represents user-specific context, and how knowledge systems are kept fresh. The technical question this pillar answers: when retrieval-augmented generation is not enough, what comes next?



The relevant cloud-native stacks are Vertex Vector Search and BigQuery ML on Google, Amazon Neptune (graph) and Kendra on AWS, AI Search Hybrid and Cosmos DB on Azure, the Vector Stores API and File Search on OpenAI, and Anthropic's contextual-retrieval patterns. Dedicated knowledge-graph engines worth knowing: Neo4j, Microsoft GraphRAG, FalkorDB, ArangoDB. For data versioning and lineage: lakeFS, DVC (Data Version Control), and W&B Artifacts. **Key takeaway: move from simple "search" to deep relational "knowledge systems."**

IV. Multimodal Reality

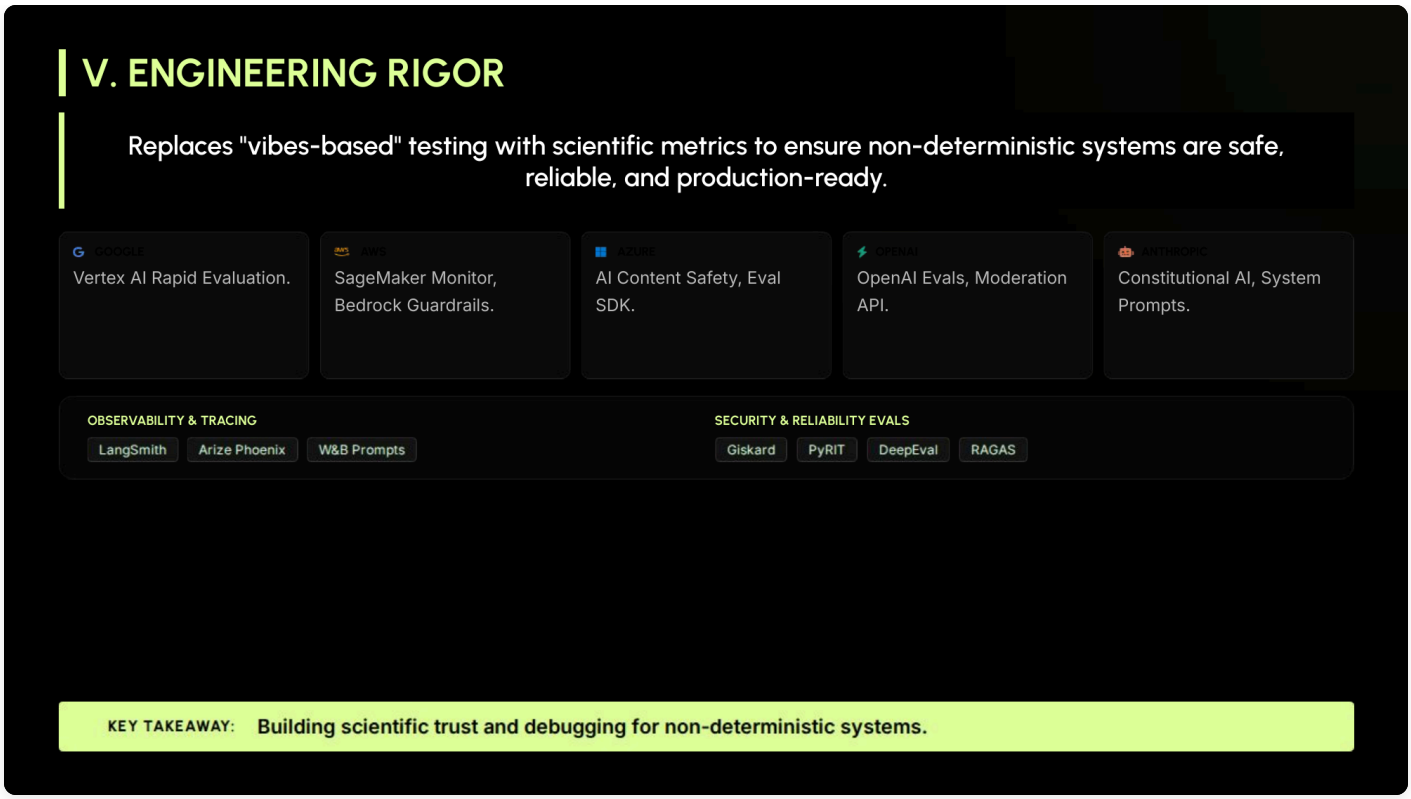
Pixels, video, spatial data, and the bridge between physical and digital. Multimodal models are no longer a curiosity; they are how AI systems perceive the world the user actually lives in. A specialist here works with vision-language models, video understanding, spatial reasoning over 3D or AR contexts, and the engineering that lets an agent both see and act. The interesting frontier is not single-frame image understanding; it is sustained multimodal interaction, where an agent watches a stream, maintains state about what it has seen, and grounds its actions in the physical environment.



Foundation-model surface area: Gemini 1.5 Pro and Vertex AI Vision API on Google, Amazon Rekognition and Multimodal Titan on AWS, Azure AI Vision and GPT-4o Vision on Azure, o1 Vision plus Whisper and DALL-E on OpenAI, Claude 3.5 Sonnet Vision API on Anthropic. Open-source spatial and vision stacks: OpenCV, Segment Anything (SAM), PyTorch Video. For digital interaction (the agent acting on screens and browsers): Playwright, WebVoyager, and the Computer Use SDK. **Key takeaway: interact with the physical, analog, and digital world in real time.**

V. Engineering Rigor

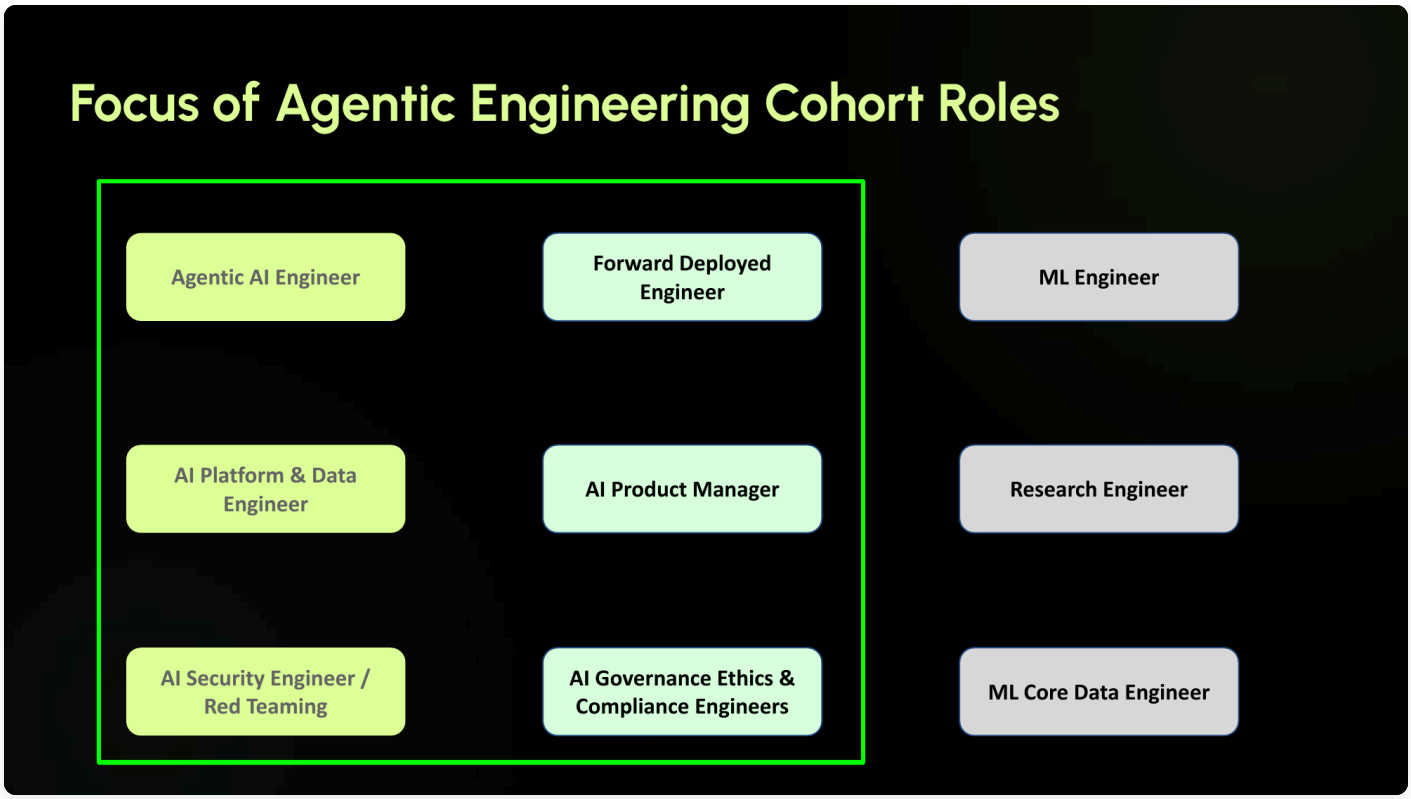
The least glamorous and arguably the most valuable. This pillar is observability, tracing, and deterministic evaluation. Concretely: building eval harnesses that catch regressions before deploy, designing deterministic test sets that survive model upgrades, instrumenting traces so you can replay an agent run and see every tool call and intermediate thought, and setting up dashboards that surface drift before users notice. The engineers who own this pillar are the ones who keep AI systems trustworthy in production. Without them, every agent is a vibe-based system held together by hope.



Cloud-native eval and safety: Vertex AI Rapid Evaluation, SageMaker Monitor with Bedrock Guardrails, Azure AI Content Safety with the Eval SDK, OpenAI Evals plus the Moderation API, and Anthropic's Constitutional AI plus system-prompt patterns. Open observability and tracing: LangSmith, Arize Phoenix, W&B Prompts. Security and reliability evals: Giskard, PyRIT, DeepEval, RAGAS. **Key takeaway: build scientific trust and debuggability for non-deterministic systems.**

The Six AI Engineering Roles

Alongside the foundational concept, each student picks a North Star role from six in-scope tracks. The role is the career identity you build the cohort around. The lesson made the in-scope versus out-of-scope split very visible:



The six in-scope roles, with the sub-competencies the lesson named for each:

- **Agentic AI Engineer** — designs and ships agentic systems end-to-end. Sub-competencies: orchestration frameworks (LangGraph, ADK, CrewAI, AutoGen), reasoning patterns (Chain-of-Thought, ReAct), tool-use design (robust APIs LLMs can call reliably), and evaluation (LLM-as-a-judge frameworks for agent reliability).
- **Forward Deployed Engineer** — embeds with customers, owns last-mile integration and bespoke deployment. Sub-competencies: solution architecting (rapidly prototyping RAG for niche domains like legal or medtech), edge AI (on-prem / on-device deployment where data cannot leave the perimeter), integration engineering (custom connectors for legacy ERP/CRM), and user feedback loops (human-in-the-loop systems for continuous refinement).
- **AI Platform & Data Engineer** — builds the foundry where AI models are developed, optimized, and deployed. Sub-competencies: vector database management (Pinecone, Milvus, Weaviate at scale), inference optimization (vLLM, TGI, GGUF/AWQ quantization), LLMOps (lifecycle automation for foundation and fine-tuned models), and GPU orchestration (Kubernetes plus Kueue/Ray).
- **AI Product Manager** — translates "what is technically possible" into "what is commercially valuable." Sub-competencies: AI unit economics (token cost vs latency vs accuracy trade-offs for ROI), UX for uncertainty (designing interfaces that handle

hallucinations and probabilistic outputs gracefully), data strategy (identifying moat-building proprietary datasets), and compliance literacy (EU AI Act and evolving regulatory frameworks).

- **AI Security Engineer / Red Teaming** — protects model weights, architecture, and sensitive data from adversarial attacks. Sub-competencies: adversarial robustness (defending against prompt injection, jailbreaking, extraction), privacy-preserving ML (Differential Privacy, Federated Learning), red teaming (automated toxicity and data-leakage testing), and supply-chain security (auditing model weights and third-party dependency risk).
- **AI Governance, Ethics & Compliance Engineer** — ensures the AI system is legally sound, morally responsible, and transparent. Sub-competencies: bias mitigation (using tools like Fairlearn to audit and correct disparate impact), explainability (SHAP, LIME for high-stakes decisions), regulatory documentation (Model Cards, transparency logs), and policy implementation (translating legal requirements into hard-coded technical guardrails).

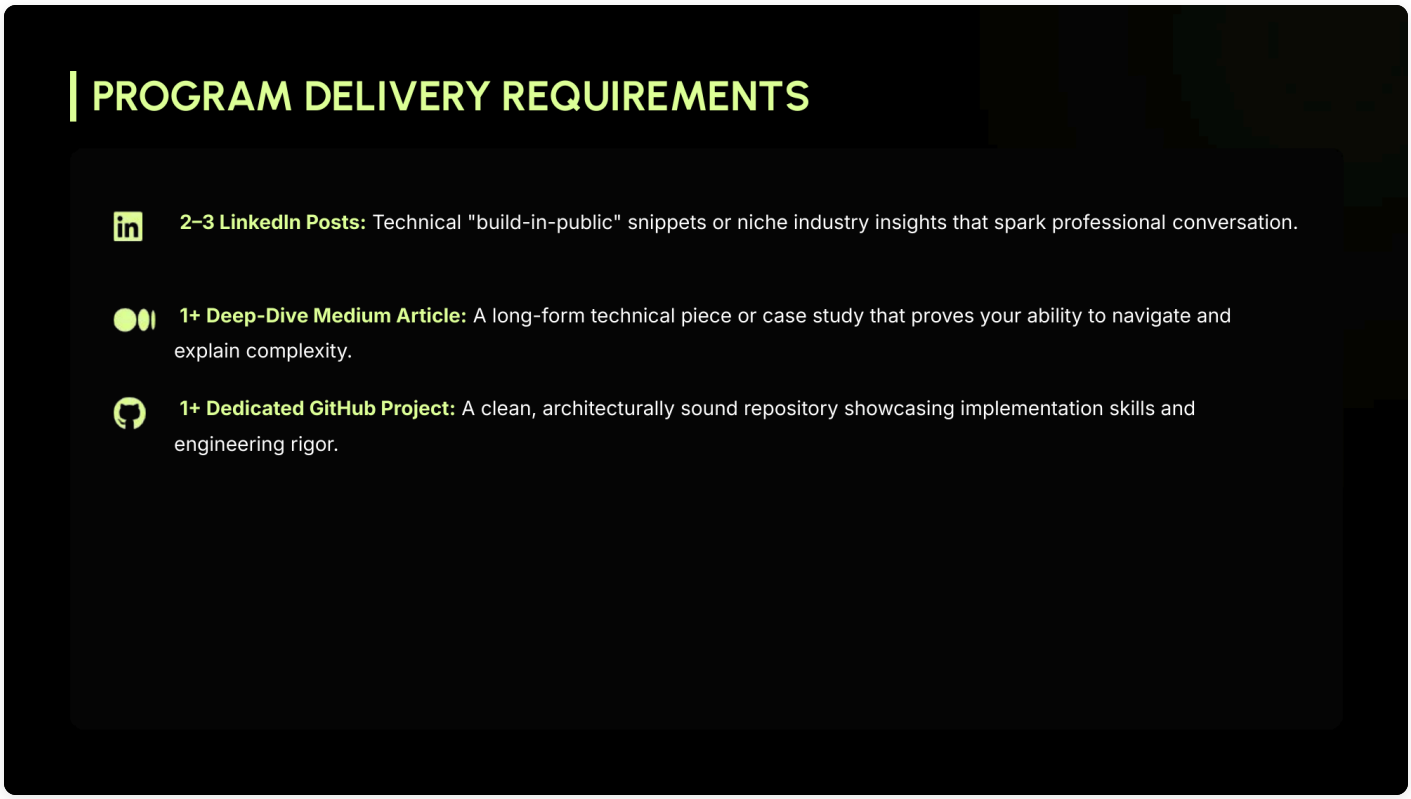
What Is Deliberately Out of Scope

Three traditional roles are explicitly excluded from this cohort: **ML Engineer**, **Research Engineer**, and **ML Core Data Engineer**. These are the "build the model from scratch" roles, the world of training pipelines, dataset curation for pretraining, novel architecture research (scaling laws, beyond-Transformer architectures like Mamba), and the petabyte-scale data work that supports it (feature stores, lineage, unstructured pipelines).

The reasoning is strategic, not dismissive. The cohort's thesis is that the leverage in the current AI labor market has shifted. Foundation models are increasingly commoditized at the production layer; the differentiated, high-paying work is in *applying* those models, *orchestrating* them into agentic systems, *deploying* them into real workflows, and *governing* their behavior. Building a foundation model from scratch is a multi-hundred-million-dollar exercise concentrated at a handful of labs. Building the systems on top of those foundation models is where most teams need talent right now. The cohort optimizes for that demand curve.

The Deliverable Triad: LinkedIn, Medium, GitHub

Every student in the cohort ships three categories of public artifact:



- **2–3 LinkedIn posts** — short, technical, build-in-public snippets. Not motivational quotes. Specific things you learned, broke, or shipped, written for a peer audience.
- **1+ Medium article** — long-form deep dive. A case study, a teardown, an architecture write-up. Something a hiring engineer could read and form a real opinion of your craft.
- **1+ GitHub project** — a clean, architecturally sound repository. Not a notebook dump. Code structured the way you would structure it on a team.

The framing the program uses is direct: **Visibility follows publication. Opportunities follow authority.** You do not become a known engineer in a niche by being good in private. You become known by writing what you know down, in public, repeatedly, in a place searchable by the people who hire.

The default audience for these artifacts is FAANG+ recruiters, but the same artifact strategy serves anyone whose attention you want — investors, design partners, future co-founders, conference programmers. Publishing is not a marketing afterthought

tacked onto a technical program. It is part of the curriculum because the cohort treats career outcomes as a problem to be engineered, not a thing that just happens to people who are good enough.

Published Presence and Passionate Narrative

The lesson framed the personal brand around a deliberate split. **Published Presence** is the unified profile strategy that converts your projects into public artifacts and positions you as a specialized expert legible to recruiters and engineering leaders. **Passionate Narrative** is the storytelling layer on top of the artifacts: continuous discovery, build-in-public snippets, evidence of authentic curiosity. The two work together. Posts and repos without a narrative are just noise; a narrative without artifacts is just claims. Both halves are required.

Week 1 Assignments

Three deliverables anchor Week 1, and they fit together as a unit: pick a role, pick a niche, and start working in the actual tooling.

1. Define Your North Star

Take the six in-scope roles. Analyze each one against your personal traits — what you are good at, what you tolerate, what bores you — and against external market demand. Declare a chosen role. The output is a written rationale, not just a label. The point of the exercise is to force a defensible choice rather than a fuzzy "I think maybe the agentic one."

2. Claim Your Territory

Intersect personal passion with the five foundational concepts. The SME niche is the concept you commit to becoming credibly deep in. Combined with the North Star role, this is the (role, concept) pair that defines what you publish about, what you build, and what you become legible for.

3. Upgrade the Gourmet Party Planner App

The cohort provides a baseline agentic application called *The Gourmet Party Planner*. Your job is to deconstruct it, fine-tune the sub-agent configurations, and in the process master the toolchain — Cursor and AntiGravity. This is the hands-on counterweight to the two strategic assignments above. The strategic ones force you to choose; this one forces you to ship. By the end of the week you should have a working modified agent and the muscle memory of the dev environment you will use for the rest of the cohort.

A worked example prompt (credit: Malik Mokhtar)

To make the assignment concrete, here is one student's prompt for upgrading the baseline app into a multi-agent system called **AGEP** (Autonomous Gourmet Event Planner). It is a useful reference for the level of specificity the assignment rewards: explicit roles, explicit temperatures, explicit failure paths, explicit test scenarios.

> **Role:** Senior AI Platform Engineer **Task:** Develop "AGEP" (Autonomous Gourmet Event Planner) using Google ADK
>
> Act as a Senior AI Platform Engineer. Build a Python-based multi-agent system named AGEP using the Google Agent Development Kit (ADK). This system must implement a strict Plan-Act-Reflect loop with specialized sub-loops for budget correction and hallucination prevention.
>
> **1. System Architecture & Agent Roles** Define four distinct agents using ADK abstractions: - **ArchitectAgent (The Planner):** (Temp 0.7) Responsible for strategic decomposition of user intent into a JSON-based menu and task roadmap. - **ExecutorAgent (The Act):** (Temp 0.1) Uses tools (Search, simulated Grocery/Nutrition APIs) to ground the plan in real-world pricing and availability. - **CriticAgent (The Reflector):** (Temp 0.0) Performs logical validation. Triggers the Correction Loop if budget or macro constraints are violated. - **VerifierAgent (The Safety):** (Temp 0.0) A "Zero-Trust" agent. Triggers the Hallucination Prevention Loop by cross-referencing ingredients against a Nutrition/Allergy DB.
>
> **2. Logic & Flow Requirements** - **Iterative Loop:** Implement a controller that allows for a maximum of 5 iterations before failing. - **Correction Loop:** If the CriticAgent rejects the plan (e.g., budget overage), it must pass specific "Delta-

> Instructions" back to the Architect for a re-plan. - **Hallucination Prevention:** The VerifierAgent must audit every ingredient. If a safety violation is found (e.g., Architect suggesting "almonds" for a nut-allergy guest), the plan is immediately rejected and sent back to the Architect. - **Human Escalation:** Throw a `ConstraintConflictError` if constraints are mathematically impossible (e.g., $10 budget for 20 people).
>
> **3. Implementation Details** - **Language:** Python 3.10+ - **Framework:** Google Agent Development Kit (ADK) - **Code Style:** Use strict Type Hinting, Clean Architecture (separate files for `agents.py`, `tools.py`, `state.py`, and `main.py`) - **Naming Conventions:** snake_case for functions/variables, PascalCase for classes - Focus on high readability for students.
>
> **4. Test Scenario** - **Input:** "Dinner for 6, $200 budget, 1 Vegan, 1 Nut-Allergy. Use Salmon & Quinoa." - **Expected Behavior:** System should catch if Salmon prices exceed budget or if a recipe accidentally includes a nut-based filler, and loop until a valid plan is formed.

Notice what this prompt does well. It separates strategic planning (Architect, high temperature) from grounded execution (Executor, low temperature) from validation (Critic, zero temperature) from safety (Verifier, zero temperature). It defines the loop's exit conditions, both successful and failure. It specifies a concrete test case the system has to pass, which means the prompt itself encodes its own acceptance criteria. This is the level of design intent the assignment is asking you to bring.

A reference implementation: AGEP

For an example of what the assignment looks like fully built out, see github.com/IrfanThomson/agep — a four-agent Plan-Act-Reflect dinner-event planner on Google ADK, iterated v1 → v2 (adversarial Saboteur agent) → v3 (post-loop Chef stage producing a cookable plan). The repo includes a 6-minute explainer video walking through the architecture and the loop in action:

▶ Watch the AGEP explainer (6:39)

Use it the way you would use a worked solution in a math textbook: study the structure, notice the design choices, then build your own version differently.

The three assignments are intentionally interlocking. The role tells you who you are becoming. The niche tells you what you are becoming known for. The app upgrade tells you how the work actually feels. None of them work in isolation.

The Quality Deep Work Framework

The closing framing of Lesson 1 is a three-part operating principle for how to behave during the rest of the cohort.

Own the domain. Identify the global leaders in your chosen focus area. Read what they ship, watch what they say, dissect their systems. Then form your own point of view. Not a regurgitation, a position. Engineers without a POV are interchangeable; engineers with a defensible POV are recruited.

Master the flow. Pick apart multi-agent orchestration in real systems. Find the seams. Understand specifically where hand-offs between agents fail, where context gets lost,

where retries paper over real bugs. The people who can articulate these failure modes are the people other teams want in the room when something is broken.

Founder mindset. The orchestration logic *is* the moat for an AI startup. The model is rented; the way you wire it together is yours. And from the FAANG+ angle, the same skill — the ability to articulate architectural trade-offs in agentic systems out loud, in an interview, in a design review — is what gets you hired into senior roles. Whether the goal is a startup or a senior IC seat, the underlying competence is identical: deep, deliberate work on systems that are still being figured out in public.

Where Lesson 1 Leaves You

Lesson 1 is a framing lesson. It does not teach you how to build an agent; it teaches you the frame the cohort uses to decide what is worth building, what is worth publishing, and what is worth becoming known for. The Agentic Shift is the macro thesis. The five concepts and six roles are the menu. The deliverable triad is the output channel. The Week 1 assignments are the on-ramp. The Quality Deep Work framework is the posture you bring to all of it.

The work begins now. By the time you reach the Publish milestone, the choices you made in Week 1 will already be visible in the artifacts you put your name on. Choose accordingly.